Productive Output Efficiency

A Methodology for Quantifying Individual LLM Cost-Efficiency

Noah Thomas Georgiana Chelednik

May 2026

Executive Brief (For Non-Academic Readers)

For every dollar I spend on AI subscriptions, I generate 6,241 words of productive output — domain-specific outputs, production code, published writing, research, system design. Over 33 months, $2,080 in subscriptions produced 13 million words of output including a 100,000+ line production software system, hundreds of domain-specific processed works, and multiple published papers and articles.

The AI industry has no standard metric for this measurement. Enterprises spend an average of $384,500 per year on OpenAI API costs (Zylo, 2026) and track token consumption, not productive output. Recent data shows that high AI adoption produces 2x throughput at 10x cost, with 861% increases in code churn — most AI-generated output is waste. My output persists: the code runs in production, the processed works are published, the papers are complete.

This document proposes the first standardized methodology for measuring individual output-per-dollar efficiency. It defines three metrics at increasing levels of rigor, presents my data as the first case study, and provides open-source tools for anyone to compute their own efficiency. The methodology establishes a measuring stick; broader adoption establishes the benchmark.

Headline numbers:

Metric	Value
Productive Output per Dollar (POD, raw)	6,241 words/$
Deliverable-Linked Output per Dollar (DLOD)	640 words/$
Quality-Adjusted POE (balanced estimate)	1,467 words/$
Quality-Adjusted POE (stress test floor)	31.6 words/$
Total cost over 33 months	$2,080
Total productive output	12,982,012 words
Input-to-output leverage	6.3:1

Executive Summary

This paper proposes a standardized methodology for quantifying the cost-efficiency of individual LLM-directed work. It addresses a gap: the AI industry measures token consumption, API spend, and speed improvements, but no existing metric captures productive output per dollar at the individual practitioner level. This gap has produced the "tokenmaxxing" crisis — companies incentivizing AI usage volume while costs balloon and output quality degrades.

The methodology defines three metrics at different auditability tiers:

Tier 1 — Productive Output per Dollar (POD): Total AI-generated output words divided by total subscription cost. Fully auditable from platform exports and billing records. Makes no quality claims.

> **POD baseline: 6,241 words per dollar.** 12,982,012 output words / $2,080 total cost. Auditable from complete ChatGPT and Claude exports processed through open-source scripts.

Tier 2 — Deliverable-Linked Output per Dollar (DLOD): Output from conversations traceable to named, verified deliverables, divided by total cost. More conservative — only counts output with external verification.

> **DLOD (all attributed): 640 words per dollar.** 1,330,736 words from 188 project-linked conversations. Attribution coverage: 11.5% of conversations, 15.7% of output. This is a deliberate lower bound.

Tier 3 — Productive Output Efficiency (POE): Output adjusted by observable quality proxies, divided by cost. Reported as a range across five configurations, not a single estimate.

> **POE range: 31.6 to 6,241 words per dollar.** From stress-test floor (maximum skepticism, including hardware amortization) to unadjusted POD. Balanced estimate: 1,467 words per dollar.

This paper proposes a methodology and presents its first application. One case study establishes the method. A hundred practitioners establish a benchmark. The methodology is designed so that anyone with platform exports and billing records can compute their own POD and compare.

---

1. The Measurement Gap

1.1 What the Industry Measures

The AI industry tracks three categories of metrics, none of which captures individual output efficiency.

Infrastructure metrics measure the supply side: tokens per watt (Jensen Huang, GTC 2026), cost per million tokens, inference throughput. These tell providers how to price their services. They tell users nothing about what they get for their money.

Enterprise volume metrics measure adoption: average organization API spend ($384,500/year, Zylo April 2026), token consumption growth (13x since January 2025), per-engineer tool costs ($500–2,000/month). These tell CFOs how much AI costs. They do not tell them what it produced.

Productivity proxies measure speed: tasks completed 25.1% faster (Harvard Business School), 55% faster with GitHub Copilot, 3.6 hours saved per week per developer. These suggest AI makes work faster. They do not measure whether faster work produces proportionally more output per dollar, or whether the output persists.

A rigorous counterpoint exists. The METR randomized controlled trial (July 2025) found that AI tools made experienced developers 19% *slower* on familiar codebases — while those same developers believed they were 20% faster. The gap between perceived and measured productivity is itself evidence that the current metrics are inadequate.

1.2 What Nobody Measures

No existing metric answers: "For each dollar a practitioner spends on AI, how many words of productive output do they generate?"

This is not a trivial omission. Without output-per-dollar measurement:

- Enterprises cannot distinguish between practitioners who generate lasting deliverables and those who generate churn.
- Individuals cannot demonstrate their efficiency relative to alternatives.
- The industry cannot identify what practices produce the highest return on AI investment.
- Cost reduction defaults to token minimization rather than output maximization.

1.3 The Tokenmaxxing Crisis

The absence of output-efficiency metrics has produced a measurable crisis. "Tokenmaxxing" — the practice of treating AI token consumption as a productivity proxy — is now documented across major technology companies.

Faros AI's March 2026 report, drawing on data from 22,000 developers across 4,000 teams, found:

- **Code churn increased 861%** under high AI adoption. Most AI-generated code is deleted within weeks.
- **Cost per merged pull request: $0.28 at low AI usage versus $89.32 at high usage.** A 319-fold increase in cost for 2x throughput — an efficiency ratio of 0.2x.
- **Bugs per developer increased 54%.** Median code review time increased 5x.
- Real-world code acceptance rates drop to 10–30% when accounting for post-merge churn, versus the 80–90% acceptance rates reported at merge time.

Amazon employees have admitted to using AI tools unnecessarily to inflate internal usage scores. Microsoft paused Claude Code usage after examining cost. Nvidia's VP of Applied Deep Learning,

Bryan Catanzaro, stated publicly that "the cost of compute is far beyond the costs of the employees" on his team.

The industry is measuring the wrong thing. Token consumption rewards volume. What is needed is a metric that rewards productive output per unit of cost. This paper proposes such a metric.

---

2. Metric Definitions

2.1 Productive Output per Dollar (POD)

$$\text{POD}_{\text{words}} = \frac{\sum \text{assistant output words}}{\text{total subscription cost (\$)}}$$

POD is the simplest metric. It counts all AI-generated output words from complete platform exports and divides by verified subscription cost. It is fully auditable: the numerator comes from export data processed through open-source scripts; the denominator comes from billing records.

POD makes no quality claims. It measures scale — how much output a practitioner extracts from their investment.

2.2 The Verbose Output Objection

A user who prompts "write me 10,000 words on nothing" and receives 10,000 words has a high POD. This is acknowledged, and it is by design.

POD is the floor metric — the denominator against which quality-adjusted metrics are measured. A high POD paired with a low DLOD (few deliverables) indicates verbose, unproductive usage. A high POD paired with a high DLOD indicates scaled, productive usage. The diagnostic value is in the *relationship* between the tiers, not in any single number.

The paper reports POD because it is auditable and because the industry currently has no metric at all. A naive metric is better than no metric, and Tiers 2 and 3 address naiveté.

2.3 Deliverable-Linked Output per Dollar (DLOD)

$$\text{DLOD} = \frac{\sum \text{assistant output words from project-linked conversations}}{\text{total subscription cost (\$)}}$$

DLOD restricts the numerator to output from conversations traceable to named deliverables with external verification. Only conversations matched to projects via keyword attribution (with confidence levels) are counted. The rest are excluded — not assumed productive, not assumed waste. Excluded.

This produces a conservative metric. In the case study, 11.5% of conversations and 15.7% of output words are attributed. The remaining 84.3% of output likely includes substantial productive work (research, learning, exploration) that cannot be traced to a specific deliverable. DLOD accepts this loss of coverage in exchange for defensibility.

DLOD is reported at three confidence thresholds:

- **High confidence only:** project name appears explicitly in conversation title (59 conversations)
- **High + medium:** strong keyword overlap (78 conversations)
- **All attributed:** any project linkage (188 conversations)

2.4 Productive Output Efficiency (POE)

$$\text{POE} = \frac{\sum(\text{conversation output words} \times Q_i)}{\text{total subscription cost (\$)}}$$

POE adjusts output by a per-conversation quality coefficient Q_i, computed from observable proxies:

1. **Conversation outcome weight:** Conversations ending in explicit positive feedback (converged) receive full weight. Conversations with 3+ corrections or explicit frustration receive a discount. Conversations ending normally (standard completion) receive an intermediate weight.
2. **Deliverable linkage modifier:** Conversations with project attribution receive full weight. Unattributed conversations receive a discount.
3. **Branching modifier (ChatGPT):** Conversations with regeneration/branching receive a discount, since branching indicates discarded output.

POE is reported as a **range across five configurations**, not a single value:

Configuration	Label	Description
Q_{unit}	No adjustment	All weights = 1.0. Equivalent to POD.
Q_{high}	Optimistic	Moderate discounts for unlinked and frustrated
Q_{mid}	Balanced	Standard discounts across all proxies
Q_{low}	Pessimistic	Aggressive discounts; unlinked output at 0.2
Q_{floor}	Maximum skepticism	Unlinked and branched output zeroed

The reader selects their trust level. The paper does not claim a "correct" Q.

2.5 What These Metrics Cannot Compare

These metrics are designed for **individual human-directed LLM use** — a practitioner working interactively with a chatbot or CLI tool on subscription pricing. They are not comparable to:

- **Enterprise automated pipelines** ($384,500/year API spend serves hundreds of users and automated workflows — a different category of usage)

- **Per-API-call cost efficiency** (API pricing charges per token; subscription pricing is flat rate — the economics differ structurally)
- **Team-level productivity** (POD measures individual output, not organizational throughput)

Cross-individual comparison requires matched domains and similar deliverable types. A developer's POD is not directly comparable to a writer's POD in absolute terms, though the methodology applies to both.

2.6 Units

Words are the primary unit. Words are human-readable, cross-model comparable, and do not depend on tokenizer choice. Tokens are reported as a secondary unit. Both are computed from the same export data.

---

3. Cost Accounting

3.1 Subscription Cost Reconstruction

Total cost is computed from billing records:

Period	Platform	Plan	Monthly Cost	Months	Subtotal
Aug 2023 – Apr 2026	ChatGPT	Plus	$20	33	$660
Mar 2024 – Nov 2025	Claude	Pro	$20	21	$420
Dec 2025 – Apr 2026	Claude	Max	$200	5	$1,000
				Total	**$2,080**

Months with overlapping subscriptions (March 2024 onward) are accounted by summing both costs. This is correct: the practitioner paid for both platforms simultaneously.

3.2 Inclusions and Exclusions

Included: Subscription fees only — the direct cost of AI access.

Excluded: Hardware costs (the practitioner's computer), electricity, internet service, opportunity cost of time. These are excluded because they vary across practitioners and would make the metric incomparable. The stress test adds hardware amortization to the denominator to show the effect of inclusion.

3.3 Subscription Versus API Pricing

Subscription plans charge a fixed monthly fee for unlimited (or rate-limited) usage. This creates a structural incentive: the efficient practitioner extracts maximum productive output from a fixed cost. Under API pricing, each additional token costs money, creating the opposite incentive toward minimization. POD is most meaningful under subscription pricing, where the denominator is fixed and the numerator is the variable the practitioner controls.

---

4. Output Measurement

4.1 Counting

Output words and tokens are computed from complete platform exports using the open-source analysis scripts in this repository. ChatGPT exports provide per-message text via the `content.parts` field in the conversation tree. Claude exports provide per-message text via `content` blocks of type `text`. Both are processed into normalized CSVs with `word_count` and `token_count` columns per message.

4.2 Conversational Versus Agentic Modalities

The Claude export includes both web/app conversations and Claude Code CLI sessions. Claude Code sessions are identifiable by the presence of `tool_use` content blocks (Read, Edit, Bash tool calls) and are characteristically token-heavy.

Claude Code usage began in approximately December 2025 and accounts for the majority of the February 2026 output spike (6,313 messages in 108 conversations). This agentic modality produces more tokens per session but also generates verified code output.

The case study reports both blended POD (all sessions) and modality-estimated POD. The modality split is estimated from CSV patterns (message count, file path references, code-heavy content), not from ground-truth session metadata. This limitation is stated explicitly.

4.3 Exclusions

Multimodal output (images, files) is excluded from word counts. Reasoning tokens (thinking/chain-of-thought content not shown to the user) are excluded. Only visible assistant text output is counted. System prompts and hidden context are excluded.

---

5. Quality Proxies

5.1 Why Direct Quality Measurement Is Excluded

Whether a line of code is good, a document is accurate, or a paragraph is well-written cannot be determined from export data alone. Quality assessment is domain-specific, subjective, and unfalsifiable at scale. The methodology does not attempt it.

Instead, it uses observable proxies that correlate with output utility. These proxies are imperfect. They are also better than nothing, and their imperfection is addressed through the range-based reporting of POE.

5.2 Proxy 1: Conversation Outcome

Conversations with 5+ messages are classified by their terminal signals:

- **Converged:** Last user message contains explicit positive feedback (6.8% of qualifying conversations)
- **Frustrated:** 3+ correction-pattern messages, or explicit frustration in the last message (9.4%)
- **Standard completion:** Last message is from the assistant with no explicit signal (82.0%)
- **Neutral:** Other (1.8%)

Standard completion is the dominant category because most successful interactions simply end without explicit feedback. It should not be read as either positive or negative.

5.3 Proxy 2: Deliverable Linkage

Conversations linked to verified deliverables (via keyword attribution) receive full weight. Unattributed conversations receive a configurable discount. The discount acknowledges that unattributed output may be productive (research, learning, exploration) but cannot be verified from the data.

5.4 Proxy 3: Output Persistence

Code in production, published works, and completed projects constitute external evidence that output was productive. This evidence is documented in the deliverable inventory but cannot be computed automatically — it requires the practitioner to maintain records.

5.5 Proxy 4: Regeneration and Branching

213 ChatGPT conversations (11.6%) contain branch points — nodes where the user regenerated a response or explored an alternative path. Branching indicates discarded output and receives a configurable discount.

5.6 Computing Q as a Range

The Q coefficient for each conversation is the product of its outcome weight, linkage modifier, and branching modifier. The overall effective Q is the weighted average across all conversations.

Five configurations are computed, from no adjustment (Q=1.0) to maximum skepticism (unlinked and branched output zeroed). The paper reports all five. The reader selects their trust level.

Configuration | Effective Q | POE (words/$) |

|:—|—:|—:| | No adjustment (Q_{unit}) | 1.0000 | 6,241 | | Optimistic (Q_{high}) | 0.4725 | 2,949 | | Balanced (Q_{mid}) | 0.2351 | 1,467 | | Pessimistic (Q_{low}) | 0.0537 | 335 | | Maximum skepticism (Q_{floor}) | 0.0091 | 56.7 |

The large drop from Q_{unit} to Q_{high} is driven by the unlinked discount: 88.5% of conversations lack project attribution, so any penalty on unattributed output has outsized effect. This is by design — the Q range shows what happens when you demand external verification for every conversation.

---

6. Case Study: Noah Chelednik

This section presents the first application of the methodology. It is a single case study, not a benchmark. The value is in demonstrating the methodology's operation, not in establishing population norms.

6.1 Data Foundation

All data derives from complete official exports: ChatGPT (August 2023 – April 2026, 19 JSON shards) and Claude (March 2024 – April 2026, including Claude Code sessions). No sampling or extrapolation. Full methodology for data extraction is documented in the companion *Quantifying LLM Practice Hours* paper and the *Deep LLM Usage Analysis*.

Metric	ChatGPT	Claude	Combined
Conversations	1,835	563	2,398
Messages	50,412	21,015	71,427
Output words	9,414,865	3,567,147	12,982,012
Output tokens	14,994,935	5,183,405	20,178,340
Input words	1,330,677	730,740	2,061,417
Active days	847	292	883

6.2 Cost

Total verified subscription cost: **$2,080** ($660 ChatGPT Plus + $420 Claude Pro + $1,000 Claude Max).

Average monthly cost: $63. This is 0.54% of what an average organization spends on OpenAI API in one year ($384,500, Zylo 2026) — though the comparison is flagged as structurally incomparable (see Section 2.5).

6.3 Deliverable Inventory

Deliverable	Type	Verification	Conversations	High Confidence
Domain AI Pipeline	Code	100,000+ lines, GitHub	81	39
Processed Domain Works	Output	hundreds of works produced	149	—
Technomirism Paper	Prose	64-page PDF, 50 citations	12	5
VCAT (Voynich Data)	Code/Data	GitHub + HuggingFace	28	12
How-To Geek Articles	Prose	Published articles	15	1
WGU Coursework	Academic	Transcripts, 23 courses	45	—
The Muser	Code	Local repository	3	2
LLM Export Analytics	Code	GitHub	4	—
Practice Hours Paper	Prose	PDF, GitHub	4	—

9 verified deliverables. 188 conversations attributed at any confidence level. 59 at high confidence.

Attribution coverage: 11.5% of conversations, 15.7% of output words. The remaining 84.3% of output includes research, exploration, personal use, and project work with ambiguous titles that the keyword classifier could not attribute. DLOD treats this as a lower bound, not a discount.

6.4 Results

Metric	Value
POD (Tier 1)	**6,241 words/** $\|\|DLOD—highconfidenceonly\|186words/$
DLOD — high + medium	360 words/$
DLOD — all attributed (Tier 2)	**640 words/** $\|\|POE—optimistic\|2,949words/$
POE — balanced (Tier 3)	**1,467 words/** $\|\|POE—pessimistic\|335words/$
POE — stress test floor	56.7 words/$
Stress test with hardware	**31.6 words/$**

Leverage ratio: 6.30x — for every word written, 6.3 words of output received.

Per-platform:

Platform	Cost	Output Words	POD	Leverage
ChatGPT	$660	9,414,865	14,265	7.08x
Claude	$1,420	3,567,147	2,512	4.88x

ChatGPT's higher POD reflects its lower subscription cost ($20/month for 33 months). Claude's lower POD reflects the $200/month Max plan during the most productive period. The Max period (December 2025 – April 2026) is when the major AI pipeline was built — the most valuable output was produced during the lowest-POD months, illustrating why POD alone is insufficient and DLOD/POE are necessary.

---

7. Industry Comparison

7.1 What Can and Cannot Be Compared

Enterprise aggregate API spend is not comparable to individual subscription use. An organization spending $384,500/year on API serves automated pipelines, customer-facing products, and hundreds of employees. That is a different category of expenditure than an individual's $63/month subscription. This paper does not claim "I'm 500x more efficient than enterprise." It notes the scale difference for context only.

Per-engineer AI tool costs ($500–2,000/month) are more comparable on the cost side, but the output side is unavailable — no published data reports per-engineer productive output. The gap is the point: this paper proposes the metric that would fill it.

7.2 The Tokenmaxxing Contrast (Directly Comparable)

The strongest industry comparison comes from Faros AI's per-developer data, which measures the same kind of individual-level efficiency this paper targets.

Metric	Industry (Faros AI)	Case Study
Cost per output unit	$89.32/merged PR (high AI)	$231/deliverable (but each "deliverable" is an entire project)
Output persistence	861% code churn increase	Code persists: 100,000+ lines in production
Efficiency ratio	0.2x (2x throughput at 10x cost)	~1.0x (output scales with cost)
Code quality	Bugs +54%, review time +5x	423 tests, production deployment

The unit mismatch between "merged PR" and "deliverable" is important. A single a major AI project deliverable encompasses hundreds of PR-equivalent code changes. On a per-PR-equivalent basis, the case study's cost would be orders of magnitude lower than $89.32.

7.3 AI Code Generation Share

Source	AI-Generated Code Share
Industry average (Feb 2026)	26.9%
GitHub Copilot users	46%
Case study (Noah Chelednik)	100%

The case study operates at 100% AI code generation through directed collaboration — a fundamentally different model from AI-assisted human coding. All code is generated by the AI; the practitioner provides architecture, constraints, review, and direction.

7.4 Speed Impact

The METR RCT found AI tools made experienced developers 19% slower on familiar codebases. The case study produced ~1,123 lines of production code per day during the a major AI project build, versus the industry average of ~125 lines/day — a 9x multiple.

These are not directly comparable (RCT versus observational; AI-assisted versus AI-primary). The directional contrast is nonetheless striking: the same technology that slowed down developers in a controlled study produced 9x baseline output when used as the primary generator rather than an assistant.

---

8. Sensitivity Analysis

8.1 POE Sensitivity Table

Configuration	Effective Q	POE (words/$)	% of POD
No adjustment	1.0000	6,241	100%
Optimistic	0.4725	2,949	47.3%
Balanced	0.2351	1,467	23.5%
Pessimistic	0.0537	335	5.4%
Maximum skepticism	0.0091	56.7	0.9%

The effective Q drops sharply because 88.5% of conversations are unattributed. Under any configuration that penalizes unattributed output, the majority of words are discounted. This is conservative by design: the methodology prefers undercounting to overclaiming.

8.2 Cost Sensitivity

Adding hardware amortization ($50/month × 33 months = $1,650) to the denominator:

Metric	Standard Cost ($2,080)	With Hardware ($3,730)
POD	6,241	3,481
POE (balanced)	1,467	818
POE (stress test)	56.7	31.6

Even under maximum skepticism with hardware costs, the stress-test floor is 31.6 words per dollar — the absolute minimum defensible efficiency.

8.3 Attribution Sensitivity

DLOD at each confidence threshold:

Threshold	Conversations	Output Words	DLOD (words/$)
High confidence only	59	385,876	186
High + medium	78	749,678	360
All attributed	188	1,330,736	640

The 3.4x range between high-only and all-attributed reflects the classification uncertainty. The paper reports all three; the reader selects their confidence requirement.

---

9. Engagement Resolution and Efficiency

This section connects the efficiency findings to the *Technomirism* framework developed in a companion paper. The connection is presented as consistent, not causal. This section is separable from the core methodology — a reader who does not accept the Technomirism framework loses nothing from the metric definitions or computations above.

The case study data shows a distinctive interaction pattern: 57% of user messages are classified as "informational" (providing context, constraints, and specifications) rather than "directive" (issuing commands) or "interrogative" (asking questions). This pattern — providing dense context and allowing the AI to operate within that context — correlates with higher output persistence and lower correction rates.

The Technomirism framework theorizes that this pattern works because dense context provision constrains the AI's generative space, reducing waste output. A user who provides shallow context receives output from a broad, unconstrained generation space — much of which is off-target. A user who provides rich context narrows the generation space to the relevant domain, producing output that requires less correction and less regeneration.

If this theory is correct, practitioners with higher engagement resolution (denser context, more recursive iteration, greater epistemic humility about the AI's contribution) should show higher POD and higher DLOD. The case study data is consistent with this prediction, but a single case study cannot confirm it. Testing would require computing POD across multiple practitioners with varying engagement styles and correlating with their interaction patterns — a research program this paper's methodology enables but does not conduct.

---

10. Limitations

Single case study (N=1). This paper establishes a methodology and demonstrates its first application. It does not establish a benchmark. The benchmark requires multiple practitioners computing their own metrics using the same methodology. Population norms cannot be derived from a single data point.

Subscription pricing bias. The flat-rate subscription model creates a structural incentive toward volume that API pricing does not share. POD under subscription pricing rewards maximum extraction; POD under API pricing would reward minimum waste. The metric's interpretation depends on the pricing model, and this paper's case study uses subscription pricing exclusively.

No direct quality measurement. The Q coefficient uses observable proxies, not quality assessment. Code that passes tests may still be poorly designed. Outputs that are complete may still be inaccurate. The methodology identifies this as an inherent limitation and addresses it through range-based reporting rather than pretending precision it does not have.

Cross-domain incomparability. No exchange rate between code and prose is established or attempted. POD for a developer and POD for a writer are not directly comparable in absolute terms. Domain-specific POD sub-metrics are more meaningful than blended POD for cross-practitioner comparison.

Attribution coverage ceiling. 46.4% of conversations could not be classified by topic, and 88.5% could not be linked to specific deliverables. DLOD and POE are computed from the minority of conversations with attribution. This is a known lower bound, not a limitation of the metric's design — it is a limitation of keyword-based classification, which could be improved with embedding-based or LLM-assisted methods.

Output persistence is not output quality. Code in production and published works demonstrate that output was *used*, not that it was *good*. Persistence is a necessary but insufficient condition for quality.

Survivorship bias. The case study measures a practitioner who continued using AI for 33 months. Practitioners who tried AI and abandoned it would produce different metrics. The methodology does not account for selection effects in who computes their POD.

Modality estimation. The conversational/agentic modality split is estimated from CSV patterns, not from ground-truth session metadata. This estimate should be treated as approximate.

---

11. Conclusion

The AI industry spends $740 billion annually on AI-related expenditure. Token consumption has grown 13x in eighteen months. Code churn has increased 861% under high AI adoption. Enterprises report AI costs exceeding employee costs. And no standardized metric exists to measure whether any of this spending produces proportional productive output.

This paper proposes such a metric. Productive Output per Dollar (POD) is auditable from platform exports. Deliverable-Linked Output per Dollar (DLOD) is conservative and externally verifiable. Productive Output Efficiency (POE) adjusts for observable quality proxies and is reported as a range.

The first case study produces a POD of 6,241 words per dollar, a DLOD of 640 words per dollar, and a POE range of 31.6 to 6,241 words per dollar. These numbers are a single data point, not a benchmark. The benchmark requires broader adoption.

The methodology is open-source and reproducible. Anyone with ChatGPT or Claude exports and a record of their subscription costs can compute their own POD in under an hour using the companion scripts. The invitation is open: compute your own efficiency, publish the result, and contribute to the benchmark this methodology is designed to establish.

One case study defines the measuring stick. A hundred practitioners calibrate it.

Appendix A: Platform-Specific Data

Metric	ChatGPT	Claude
Conversations	1,835	563
Messages	50,412	21,015
Output words	9,414,865	3,567,147
Output tokens	14,994,935	5,183,405
Input words	1,330,677	730,740
Cost allocated	$660	$1,420 \|\| POD(words/)$
Leverage ratio	7.08x	4.88x
Active days	847	292

Appendix B: Formulas

$$\text{POD}_{\text{words}} = \frac{\sum_{m \in \text{assistant}} w_m}{C_{\text{total}}}$$

$$\text{DLOD}_{\theta} = \frac{\sum_{c \in A_\theta} \sum_{m \in c, \text{assistant}} w_m}{C_{\text{total}}}$$

where A_θ is the set of conversations attributed at confidence threshold θ.

$$\text{POE}_k = \frac{\sum_c Q_{c,k} \cdot \sum_{m \in c,\text{assistant}} w_m}{C_{\text{total}}}$$

where $Q_{c,k}$ is the quality coefficient for conversation c under configuration k:

$$Q_{c,k} = f_{\text{outcome}}(c,k) \times f_{\text{linkage}}(c,k) \times f_{\text{branch}}(c,k)$$

Stress test cost: $C_{\text{stress}} = C_{\text{total}} + H \cdot T$, where H is monthly hardware amortization and T is total months.

Appendix C: Cost Log Template

Practitioners should document their subscription history in this format:

```
[
  {
    "platform": "chatgpt | claude | other",
    "plan": "plan name",
    "monthly_cost": 20,
    "start": "YYYY-MM",
    "end": "YYYY-MM"
  }
]
```

Appendix D: Deliverable Inventory Template

```
[
  {
    "name": "Project Name",
    "type": "code | prose | data | academic | other",
    "description": "One-line description",
    "verification": {
      "method": "repository | published_document | published_articles | transcript",
      "artifact": "URL or description of verifiable artifact"
    },
    "conversations_attributed": 0,
    "high_confidence": 0,
    "active_period": "YYYY-MM to YYYY-MM"
  }
]
```

Appendix E: Reproduction

All scripts are located in `analysis/deep-analysis/scripts/`:

1. `compute_pod.py` — Computes POD from normalized CSVs and cost log
2. `compute_dlod.py` — Computes DLOD from project attributions and cost log
3. `compute_poe.py` — Computes POE range from quality proxies and cost log
4. `compare_benchmarks.py` — Compares results to curated industry benchmarks

Prerequisites: Python 3.10+, pandas, tiktoken (optional). All scripts use argparse with sensible defaults.

To compute your own POD: 1. Export your ChatGPT and/or Claude data via platform settings 2. Run the normalization scripts to produce message CSVs 3. Create a `cost_log.json` with your subscription history 4. Run `compute_pod.py --chatgpt <csv> --claude <csv> --costs <cost_log.json>`